

QF-AgentOS

An honest benchmark and evidence harness for quantum finance

QF-AgentOS contributors

Version 0.1.0 — 2026-07-16

Contents

1	Overview	4
1.1	The headline result	4
1.2	Install	5
1.3	Quickstart	5
1.4	Problem families	5
1.5	What actually runs	5
1.6	Real quantum hardware	5
1.7	Research-grade extras	6
1.8	Surfaces	6
1.9	The agent team	6
1.10	Human-control model (autonomy levels)	7
1.11	What this can and cannot tell you	7
1.12	Configuration	7
1.13	Project layout	7
1.14	Documentation	7
1.15	Disclaimer	8
1.16	License	8
2	Findings — does quantum help?	9
2.1	The leaderboard	9
2.2	Why this is trustworthy	9
2.3	Why quantum loses — per family	10
2.4	Real quantum hardware (IBM)	10
2.5	Research-grade extras	11
2.6	Limitations — what these findings do <i>not</i> prove	11
2.7	Reproducing this	11
3	Problem families	13
3.1	collateral_allocation	13
3.2	payment_routing	13
3.3	settlement_netting	14
3.4	fraud_detection	14
3.5	rfq_fill	15
3.6	Adding a family	15
4	CLI reference	16
4.1	solve	16
4.2	explain	17
4.3	plan	17
4.4	arena	17
4.5	estimate	17
4.6	simulability	17
4.7	backends	17
4.8	skills	18
4.9	runs	18
4.10	serve	18

4.11 version	18
4.12 Running on real hardware	18
5 REST API reference	19
5.1 Endpoints	19
5.2 Authentication	19
5.3 Synchronous solve	20
5.4 Asynchronous solve (the production path)	20
5.5 Studio entry point	20
5.6 Error codes	21
5.7 QF-Studio	21
5.8 Registries	21
5.9 Docker	21
6 Configuration	22
6.1 Observability	22
6.2 Determinism & numerics	22
6.3 Simulation budgets	22
6.4 Governance / registry	23
6.5 REST API	23
6.6 Credentials (secrets)	23
6.7 Spec-level policy (per run, not env)	23
6.8 Extras	24
7 Architecture	25
7.1 High-level flow	25
7.2 The agent contract	25
7.3 The three formulations (fair comparison)	26
7.4 Backends (backends/)	26
7.5 Safety model (core/policy.py)	26
7.6 Configuration & observability	27
7.7 Governance (governance/)	27
7.8 Surfaces	27
7.9 Extending	27
8 Running on real quantum hardware	28
8.1 Prerequisites	28
8.2 Provide the credential — as an environment variable, never in a file	28
8.3 Opt in, and approve the paid run	28
8.4 What you get	29
8.5 D-Wave (annealing)	29
9 Research note — QAE & tensor networks	30
10 Part A — Quantum Amplitude Estimation for risk	31
10.1 Problem statement	31
10.2 Classical baseline	31
10.3 Quantum approach	31
10.4 Complexity analysis	32
10.5 Hardware feasibility	32
10.6 Empirical result (this implementation)	32
10.7 Open questions	32
11 Part B — Tensor-network classical-simulability baseline	33
11.1 Problem statement	33
11.2 Classical baseline	33
11.3 Method	33
11.4 Complexity	33
11.5 Empirical result	33
11.6 Caveats	34

11.7 Overall conclusion 34

11.8 References 34

Chapter 1

Overview

An honest benchmark and evidence harness for quantum finance.

Give QF-AgentOS a financial problem. A team of deterministic agents formulates it, builds **strong** classical baselines, identifies a quantum-compatible sub-problem, runs it (simulator or real QPU), **mechanically re-verifies** the result against the exact constraints, and reports — with regulatory-grade evidence — whether quantum actually helped.

The defining principle: *an agent that knows **when not** to use quantum computing is more valuable than one that always produces a circuit.*

Most quantum-finance demos are rigged: weak classical baselines, cherry-picked instances, encoding losses swept under the rug, “we ran it on hardware” hiding heavy classical post-processing. This is the opposite. **It is built so it cannot lie to you — including about quantum losing.**

1.1 The headline result

Across **5 problem families, 3 quantum techniques, and real IBM hardware**, with fair comparators and mechanical verification:

\$ qf-agent arena

7 problems · 0 quantum advantage · 1 parity · 6 classical preferred.

Family	Task	Classical	Quantum	Honest verdict
collateral_allocation	optimization	MILP 7,140	QAOA — (<i>infeasible</i>)	classical preferred
payment_routing	optimization	MILP 1,869	QAOA — (<i>infeasible</i>)	classical preferred
settlement_netting	optimization	MILP 0	QAOA 0	quantum parity
fraud_detection	classification	AUC 0.983	quantum kernel AUC 0.557	classical preferred
rfq_fill	classification	AUC 0.945	quantum kernel AUC 0.496	classical preferred

Quantum won zero times. The QAOA solutions that *did* decode were infeasible against the exact constraints; the quantum kernels performed at or below chance (0.496 is worse than a coin flip). On real IBM hardware (ibm_kingston) the circuit also decoded an *infeasible* solution — reported honestly, not hidden.

Reproduce it yourself with qf-agent arena. See [docs/FINDINGS.md](#) for the full evidence and — more importantly — *why* each one loses. (Spoiler: the encoding decides the outcome before the solver ever runs.)

That is not a bug. **That is the product.**

1.2 Install

```

pip install -e ".[all,dev]"          # classical + qiskit + pennylane + server + tests
# minimal (classical only):          pip install -e .
# gate-model simulation:             pip install -e ".[qiskit]"
# REST API + Studio:                pip install -e ".[server]"
# real hardware:                    pip install -e ".[ibm]" / ".[dwave]"
# registries:                       pip install -e ".[mlflow]" / ".[postgres]"

```

Requires Python ≥ 3.11 .

1.3 Quickstart

```

qf-agent solve examples/collateral-allocation.yaml # full pipeline + evidence bundle
qf-agent arena --out arena/                       # benchmark every family (the leaderboard)
qf-agent backends                                # what's actually available right now
qf-agent estimate --qubits 4                      # amplitude estimation vs classical MC
qf-agent simulability examples/collateral-allocation.yaml # tensor-network classical check
qf-agent serve                                    # REST API + Studio on :8000

```

Full command reference: [docs/CLI.md](#).

1.4 Problem families

The pipeline is problem-agnostic (a ProblemDomain abstraction); a family plugs into the same agents, backends, verification, audit, and governance. Two task types — optimization (MILP + QAOA) and classification (classical baselines + quantum kernels) — are selected automatically.

- **collateral_allocation** — minimise posting cost s.t. coverage, HQLA, concentration.
- **payment_routing** — generalized assignment: route transactions at least expected cost.
- **settlement_netting** — RTGS gridlock resolution: settle a batch s.t. per-participant liquidity.
- **fraud_detection** — quantum fidelity kernel vs RBF, same learner, temporal holdout.
- **rfq_fill** — will a request-for-quote fill? (shares the quantum-kernel base).

Details + encoding losses: [docs/PROBLEM-FAMILIES.md](#).

1.5 What actually runs

For every optimisation problem the pipeline compares *like with like*:

Layer	Method	Backend
Full problem — lower bound	Continuous LP relaxation	SciPy/HiGHS
Full problem — fair comparator & recommendation	Binary MILP (exact)	SciPy/HiGHS
Research instance — classical optimum	Binary MILP (reduced)	SciPy/HiGHS
Research instance — QUBO ground truth	Brute-force enumeration	CPU
Research instance — strong heuristic	Simulated annealing	CPU
Research instance — quantum	QUBO → Ising → QAOA	statevector sim / real QPU

The **Verification** agent re-checks every decoded quantum solution against the full exact constraint set — so relaxation artifacts are caught, never hidden.

1.6 Real quantum hardware

IBM is validated end-to-end (a clean 9-step run on `ibm_kingston`); **D-Wave annealing is wired and mock-tested**. Both are credential-gated behind autonomy **L3 + explicit approval + budget**, and fall back

to the simulator with a recorded reason.

```
execution_policy:
  gpu_backend: ibm      # or: dwave   (default: sim)
  autonomy_level: L3
  max_gpu_budget_usd: 5
```

Runbook: [docs/real-gpu.md](#). Credentials are read from the environment as secrets and are never logged or written into evidence.

1.7 Research-grade extras

- **Quantum Amplitude Estimation** (`qf-agent estimate`) — MLAE with a real Grover operator vs exact summation + Monte Carlo. Verdict: the quadratic speedup is *proven but asymptotic*, $O(2^m)$ state-prep-bottlenecked, early-fault-tolerance gated.
- **Tensor-network baseline** (`qf-agent simulability`) — is the QAOA circuit classically simulable by an MPS? Entanglement entropy, bond dimension, MPS fidelity.

Methodology + citations: [docs/research-note-quantum-extras.md](#).

1.8 Surfaces

Surface	How
CLI	<code>qf-agent ...</code> — 11 commands (reference)
SDK	<code>from qf_agentos import load_spec, solve</code>
REST API	<code>qf-agent serve</code> — <code>sync /solve</code> , <code>async /jobs</code> (reference)
QF-Studio	web UI at / (bundled, no build step)
QF-Studio (React)	richer SPA in <code>studio-react/</code>
Docker	<code>docker compose up --build</code>

```
from qf_agentos import load_spec, solve

ctx = solve(load_spec("examples/collateral-allocation.yaml"))
print(ctx.state.audit.rendered)           # the FINAL DECISION block
print(ctx.state.bundle.manifest["evidence_digest"]) # deterministic per spec+seed
```

1.9 The agent team

1. **Requirements** — validate the Finance IR, discover missing constraints.
2. **Formulation** — enumerate LP / MILP / QUBO / Ising and what each can/cannot represent.
3. **Classical Baseline** — serious LP + MILP via HiGHS (never a strawman). 4/5. **Hardware Planner** — reduce to a qubit-sized instance, discover backends, estimate depth/cost, and decide whether quantum is warranted — **abstention is first-class**.
4. **Execution** — the instance ladder: MILP → QUBO-exact → SA → QAOA, under the policy engine.
5. **Verification** — objective recomputation, exact constraint checks, quantum-contribution accounting.
6. **Quantum-Advantage Auditor** — an honest outcome category + the FINAL DECISION block.
7. **Governance** — manifest, technical report, model card, deterministic evidence digest.

1.9.1 Outcome categories

CLASSICAL PREFERRED · QUANTUM NOT FEASIBLE ON PRESENT HARDWARE · QUANTUM PARITY · QUANTUM RESEARCH CANDIDATE · QUANTUM IMPROVEMENT OBSERVED · INDEPENDENT REPRODUCTION REQUIRED · POTENTIAL OPERATIONAL ADVANTAGE

The auditor’s bias is toward classical: a quantum result must be **verified feasible** and at least match the classical optimum before it earns anything better than “classical preferred”.

1.10 Human-control model (autonomy levels)

Level	Capability	Extra gate
L0	Explain and recommend	—
L1	Generate experiment plan	—
L2	Execute simulators automatically	—
L3	Execute paid QPU jobs	budget + human approval
L4	Recommend production decisions	human approval mandatory

The system never places trades, moves money, alters limits/fraud rules, runs paid QPU jobs without approval, or claims quantum advantage without verification.

1.11 What this can and cannot tell you

Honest limitations — read these.

- **Scale ceiling.** Quantum runs on *reduced research instances* ($\leq \sim 22$ qubits). At these sizes exact classical methods win trivially. This platform **cannot** tell you where a future crossover lies at production scale; it tells you the honest answer *today*, and gives you the instrument to detect a crossover when hardware arrives.
- **The verdict is mostly decided by the encoding, not the solver.** A QUBO often cannot faithfully represent a real financial constraint at tractable penalty weights (see the settlement finding in [FINDINGS](#)). This is a structural result, not a tuning failure.
- **“Agentic” means a deterministic agent pipeline** — no LLM reasoning at runtime. Same spec + seed $\Rightarrow$ byte-identical evidence digest.
- **This is a research instrument and methodology standard**, not a trading system.

1.12 Configuration

Environment variables (prefix QF_) — full table in [docs/CONFIGURATION.md](#): QF_LOG_LEVEL, QF_LOG_FORMAT, QF_DEFAULT_SEED, QF_STATEVECTOR_QUBIT_LIMIT, QF_EVIDENCE_DIR, QF_REGISTRY_BACKEND (file/mlflow/postgres), QF_API_KEYS, and credentials QF_IBM_TOKEN / QF_DWAVE_TOKEN (secrets, never logged).

1.13 Project layout

```
src/qf_agentos/
├─ core/           # Finance IR, typed state, artifacts, policy, config, logging, workflow
├─ finance/        # 5 domains + qml/qkernel, qae (amplitude estimation), tensor_network
├─ backends/       # protocol, registry, solvers (exact/SA/QAOA), IBM/D-Wave/PennyLane
├─ agents/         # the 9 agents (one module each)
├─ arena/          # QF-Arena benchmark runner + leaderboard
├─ governance/     # evidence bundle + file / MLflow / Postgres registries
├─ skills/         # Quantum Skills plugin registry (5 skills)
├─ studio/         # bundled web UI (served at /)
├─ api.py          # FastAPI service (sync + async jobs)
├─ cli.py          # the `qf-agent` command (11 commands)
studio-react/     # richer React + TypeScript SPA (separate deployable)
docs/             # architecture, findings, CLI, API, configuration, families, research notes
examples/         # 7 runnable specs
tests/            # 229 tests, ~92% coverage, mypy --strict
```

1.14 Documentation

- [Findings](#) — the honest results, and *why* quantum loses

- [Problem families](#) · [CLI](#) · [REST API](#) · [Configuration](#)
- [Architecture](#) · [Real QPU runbook](#)
- [Research note: QAE + tensor networks](#)
- [Contributing & extension guide](#) · [Security & safety model](#) · [Changelog](#)

1.14.1 PDF (printable / shareable)

Typeset editions of everything above live in [docs/pdf/](#):

- [QF-AgentOS-Documentation.pdf](#) — the complete manual (cover, contents, every doc)
- [QF-AgentOS-Findings.pdf](#) — the results as a standalone report
- One PDF per document alongside them.

Rebuild after editing any Markdown (needs pandoc + a TeX distribution):

```
python3 scripts/build-pdfs.py
```

1.15 Disclaimer

Experimental research software. Not investment advice, not a licensed advisor, and not a production trading system. Outputs are decision-support only.

1.16 License

Apache-2.0. See [LICENSE](#) and [NOTICE](#).

Chapter 2

Findings — does quantum help?

Short answer: no. Not on any problem this platform can run, and not by a small margin. This document is the evidence, and — more usefully — the *reasons*.

Everything here is reproducible: `qf-agent arena` regenerates the leaderboard, and a regression test (`test_full_suite_has_no_quantum_advantage`) fails the build if the platform ever starts claiming otherwise.

2.1 The leaderboard

```
$ qf-agent arena
7 problems · 0 quantum advantage · 1 parity · 6 classical preferred.
```

problem	family	task	classical	quantum	verdict
abstention-demo	collateral_allocation	optimization	MILP 3,550	none	QUANTUM NOT FEASIBLE ON PRESENT HARDWARE
collateral-allocation	collateral_allocation	optimization	MILP 7,140	QAOA — <i>(infeasible)</i>	CLASSICAL PREFERRED
payment-routing	payment_routing	optimization	MILP 1,869	QAOA — <i>(infeasible)</i>	CLASSICAL PREFERRED
settlement-netting	settlement_netting	optimization	MILP 0	QAOA 0	QUANTUM PARITY
fraud-detection	fraud_detection	classification	AUC 0.9826	quantum kernel AUC 0.5568	CLASSICAL PREFERRED
rfq-fill	rfq_fill	classification	AUC 0.945	quantum kernel AUC 0.4964	CLASSICAL PREFERRED
infeasible	collateral_allocation	optimization	infeasible	none	CLASSICAL PREFERRED

Score convention: objective = lower better; AUC = higher better. Deterministic per seed.

Note what “—” means for quantum: the QAOA *did* run and *did* decode a solution — it was **infeasible** against the exact constraints, so it earns no score. That is the verification agent doing its job.

2.2 Why this is trustworthy

A negative result is only worth something if the comparison was fair. The guardrails:

- **The classical baseline is not a strawman.** Exact binary MILP via SciPy/HiGHS (plus an LP lower bound), solving these instances in ~ 1 ms. For classification, logistic regression *and* RBF kernel-ridge — with the quantum kernel judged against the **same** kernel-ridge learner, isolating the kernel.

- **Every quantum solution is mechanically re-verified** against the full exact constraint set — never trusted from the QUBO energy.
- **Statistical honesty in classification** — temporal holdout, data-leakage checks, bootstrap significance vs the RBF comparator.
- **Abstention is a first-class outcome** — the planner refuses when quantum isn’t warranted.
- **The auditor is biased toward classical** — quantum must be *verified feasible* and at least match the classical optimum to earn better than “classical preferred”.
- **Determinism** — same spec + seed $\Rightarrow$ byte-identical evidence_digest.

2.3 Why quantum loses — per family

The interesting part. **In most cases the encoding decided the outcome before the solver ever ran.**

2.3.1 collateral_allocation / payment_routing — the QUBO drops or distorts constraints

The QUBO encodes the objective plus *some* constraints via penalties/slack; others (concentration, HQLA, route capacity, approval floors) are **verification-only**. The QAOA optimum therefore breaches the real constraint set, and the verifier catches it. Classical MILP handles all constraints exactly, in milliseconds.

2.3.2 settlement_netting — a structural impossibility result

The most technically interesting finding. Liquidity ($\text{net outflow} \leq \text{balance}$) is a **packing constraint**. Settling an obligation of amount a that overshoots a payer’s headroom by δ gains reward $\sim a$ but costs only the quadratic penalty $A \cdot \delta^2$. Because δ can be arbitrarily small while a is large, **no tractable penalty weight A makes the QUBO’s ground state feasible** — you would need $A \sim \text{max_amount} \cdot \text{total}$, which swamps the objective and destroys the QAOA landscape.

An adversarial sweep of **320 random instances** confirmed it: 25–90% of QUBO ground states are liquidity-infeasible, **and every single one was caught and reported honestly** (zero laundered as feasible). Parity here means QAOA matched the MILP on a benign instance — not that it is competitive.

This is not a tuning failure. It is a property of the problem class. Some financial constraints simply do not survive QUBO encoding at usable penalty weights.

2.3.3 fraud_detection / rfq_fill — the quantum kernel doesn’t separate the data

The ZZ-feature-map fidelity kernel scores AUC **0.557** and **0.496** — the latter is *worse than a coin flip* — against classical **0.983 / 0.945** on identical data, the same learner, and the same temporal holdout. The bootstrap significance test finds no quantum improvement. There is no subtlety here: it just doesn’t work.

2.4 Real quantum hardware (IBM)

A full 9-step pipeline run on **ibm_kingston** (IBM Quantum, Open plan):

Signal	Value
Steps / errors	9 / 0
Backend	real hardware · ibm_kingston
QPU access time	3.0 s
Reached QUBO optimum	yes (hardware sampled the ground-state energy)
Quantum contribution	yes (shot distribution beat uniform random)
Decoded solution	infeasible (violated coverage)
Verdict	CLASSICAL PREFERRED

Read that carefully, because it is the whole thesis in one run: **the quantum hardware genuinely did something — it beat random sampling and found the QUBO’s optimum — and it still lost, because**

the QUBO's optimum is not a feasible answer to the real problem. Good quantum execution of a lossy encoding is still a wrong answer.

Getting there required fixing three real bugs the live run exposed (a retired IBM API channel, the free plan forbidding session mode, and a verification crash on the hardware's shot histogram). All are fixed; see [real-qpu.md](#).

2.5 Research-grade extras

2.5.1 Quantum Amplitude Estimation (qf-agent estimate)

QAE has a **proven quadratic speedup** over Monte Carlo — $O(1/\epsilon)$ vs $O(1/\epsilon^2)$ queries (Brassard et al. 2002). Our MLAE implementation recovers the exact expectation to $\sim 1e-3$ and the amplitude oracle is validated to machine precision. And yet:

- At runnable sizes, **exact summation is instant and error-free**, and classical MC at the same query budget **matches or beats** the QAE estimate.
- **State preparation costs $O(2^m)$ gates** — the binding constraint. The speedup survives only if amortised across many pricings (Stamatopoulos et al. 2020).
- A useful advantage needs **early fault tolerance** (Chakrabarti et al. 2021) — a 10+ year horizon.

Verdict: CLASSICAL PREFERRED. The speedup is real, proven, and irrelevant today.

2.5.2 Tensor-network baseline (qf-agent simulability)

Measures whether the QAOA circuit is classically simulable by an MPS. On these dense QUBOs the output states are *highly* entangled (13-qubit collateral: entropy ≈ 5.2 bits, bond dimension ≈ 43 vs an exact-rank max of 64), so an MPS gives **no compression** — reported honestly as a small-instance artefact, **not** as evidence of hardware advantage. (Exact statevector already solves it classically.)

Methodology + citations: [research-note-quantum-extras.md](#).

2.6 Limitations — what these findings do *not* prove

Honesty cuts both ways. This platform **does not** show that quantum computing is useless in finance forever. It shows:

- **At these scales** ($\leq \sim 22$ qubits, reduced research instances), classical wins — which is unsurprising, since exact methods solve these instances outright. The platform **cannot** locate a crossover at production scale; that question stays open.
- **For these encodings.** A better formulation (not a better solver) is where any future quantum result would have to come from. The settlement finding suggests some constraint classes are structurally hostile to QUBO — a testable, generalisable claim.
- **For these techniques** (QAOA, quantum kernels, QAE). Fault-tolerant algorithms are out of reach of any simulator.

What it *does* establish, reproducibly: **the popular near-term quantum-finance demonstrations do not survive a fair comparison** — and the reason is usually the encoding, not the hardware.

2.7 Reproducing this

```
pip install -e ".[all,dev]"
qf-agent arena --out arena/           # the leaderboard above
qf-agent solve examples/settlement-netting.yaml
qf-agent estimate --qubits 4          # QAE vs classical MC
qf-agent simulability examples/collateral-allocation.yaml
pytest -q                             # 229 tests, incl. the no-advantage regression test
```

Every run writes an evidence bundle (`manifest.json`, `report.md`, `model_card.md`) with a deterministic digest. Same spec + seed $\Rightarrow$ identical digest.

Research artifact — decision-support only, not investment advice.

Chapter 3

Problem families

Five families, two task types. The pipeline is problem-agnostic: a family implements a `ProblemDomain` (optimization) or a `ClassificationDomain`, and plugs into the same agents, backends, verification, audit, and governance.

Family	Task type	Quantum technique	Verdict (<i>evidence</i>)
<code>collateral_allocation</code>	optimization	QAOA	classical preferred
<code>payment_routing</code>	optimization	QAOA	classical preferred
<code>settlement_netting</code>	optimization	QAOA	quantum parity
<code>fraud_detection</code>	classification	quantum kernel	classical preferred
<code>rfq_fill</code>	classification	quantum kernel	classical preferred

Read the “encoding loss” note in each section. For the optimisation families that is what actually decides the verdict — not the solver, not the hardware.

3.1 collateral_allocation

Post the cheapest set of securities that covers a margin requirement.

Objective minimise posting cost $\sum (\text{cost_bps}/1e4) \cdot \text{value} \cdot x$. **s.t.** post-haircut coverage $\sum (1 - \text{haircut}) \cdot \text{value} \cdot x \geq \text{required_collateral}$, a minimum-HQLA floor, and per-issuer/counterparty concentration caps. $x \in \{0, 1\}$ (post the whole lot).

```
problem: collateral_allocation
constraints:
  required_collateral: 4000000
  minimum_hqla: 1000000
  concentration: { issuer: 0.40 }
inventory:
  - { id: BOND_A, issuer: UST, market_value: 2000000, haircut: 0.02, cost_bps: 5, hqla: true }
```

Classical LP lower bound + exact binary MILP (HiGHS). **Quantum** QUBO $\rightarrow$ Ising $\rightarrow$ QAOA on a reduced instance. **Encoding loss** Coverage is encoded, as a proper inequality via binary slack bits with an adaptive penalty. **Concentration and HQLA are dropped** (verification-only) — the QAOA optimum can breach them, and the verifier catches it. Example: `examples/collateral-allocation.yaml`.

3.2 payment_routing

Route each transaction to exactly one eligible route (a generalized assignment problem).

Objective minimise expected cost = processing + fixed fee + expected fraud + latency + expected decline $(1 - \text{approval}) \cdot \text{penalty}$ · **s.t.** one route per transaction, per-route capacity, network diversification, and a portfolio approval floor.

```
problem: payment_routing
routing: { decline_penalty_bps: 120, network_concentration: 0.60, min_overall_approval: 0.90 }
routes:   [{ id: R_VISA, cost_bps: 18, approval_rate: 0.94, capacity: 3, network: VISA }]
transactions: [{ id: T1, amount: 250000, eligible_routes: [R_VISA] }]
```

Classical sparse GAP MILP (HiGHS). **Quantum** one-hot assignment QUBO $\rightarrow$ QAOA. **Encoding loss** The QUBO encodes cost + the one-hot assignment penalty. **Capacity, network diversification, and the approval floor are dropped** (verification-only). Example: `examples/payment-routing.yaml`.

3.3 settlement_netting

RTGS gridlock resolution: pick a batch of queued interbank obligations to settle **simultaneously**, maximising settled value while nobody overdraws.

Objective minimise unsettled value (= maximise settled) · **s.t.** for each participant $\text{outflow}(\text{batch}) - \text{inflow}(\text{batch}) \leq \text{balance}$; optional settled-value floor.

```
problem: settlement_netting
settlement: { penalty_scale: 8.0 }
participants: [{ id: BANK_A, balance: 10 }, { id: BANK_D, balance: 60 }]
obligations: [{ id: O_AB, payer: BANK_A, payee: BANK_B, amount: 100 }]
```

The demo encodes a classic gridlock: $A \rightarrow B \rightarrow C \rightarrow A$ of 100 each, every bank holding only 10 — none can settle alone, but settling the cycle *simultaneously* nets to zero.

Classical exact MILP (HiGHS). **Quantum** QUBO with each participant's liquidity inequality **genuinely encoded** via binary slack bits (unlike the families above, which drop constraints).

3.3.1 ! The structural finding

Liquidity is a **packing constraint**, and it does not survive QUBO encoding. Settling an obligation of amount a that overshoots a payer's headroom by δ gains reward $\sim a$ but costs only $A \cdot \delta^2$. Since δ can be arbitrarily small while a is large, **no tractable penalty A makes the ground state feasible** — you'd need $A \sim \text{max_amount_total}$, which swamps the objective and destroys the QAOA landscape. A 320-instance adversarial sweep found 25–90% of QUBO ground states liquidity-infeasible; **every one was caught and reported honestly**. This is a property of the problem class, not a tuning failure.

Example: `examples/settlement-netting.yaml`.

3.4 fraud_detection

Binary classification on an imbalanced, temporally-ordered dataset.

Classical logistic regression (IRLS) + RBF kernel-ridge. **Quantum** ZZ feature-map **fidelity kernel** + the *same* kernel-ridge learner — so the comparison isolates the **kernel**, not the learner. **Method honesty** temporal holdout (no look-ahead), train-only standardisation, data-leakage checks, and a bootstrap significance test **against the RBF kernel** (not against the weakest classical model).

```
problem: fraud_detection
classification:
  target_metric: auc
  test_fraction: 0.3
  feature_budget: 4          # qubits = features fed to the quantum kernel
  bootstrap: 500
  synthetic: { n_samples: 240, n_features: 6, class_balance: 0.2 }
```

Inline features + labels (+ optional timestamps) are accepted instead of synthetic. **Encoding loss** The quantum kernel sees a *reduced* feature/sample instance (feature budget + an $O(n^2)$ sample cap) — not the full dataset the classical baselines see. Example: `examples/fraud-detection.yaml`.

3.5 rfq_fill

Will a request-for-quote fill? A market-maker problem: predict fill from features known at quote time — quoted spread, order size, volatility, response latency, counterparty tier.

Shares the entire `QuantumKernelClassificationDomain` pipeline with `fraud_detection`; only the synthetic generator differs (a $\text{spread} \times \text{size}$ **nonlinear** boundary — something a ZZ kernel could in principle exploit; the pipeline tests honestly whether it does. It doesn't: AUC 0.496, worse than a coin flip).

```
problem: rfq_fill
classification:
  target_metric: auc
  feature_budget: 4
  synthetic: { n_samples: 260, n_features: 6, class_balance: 0.35 }
```

Example: `examples/rfq-fill.yaml`.

3.6 Adding a family

1. Implement `ProblemDomain` (optimization) or subclass `QuantumKernelClassificationDomain` (classification) in `src/qf_agentos/finance/`.
2. Add its IR block + validation branch in `core/ir.py`.
3. Register it in `finance/__init__.py` (`KNOWN_PROBLEMS` + `get_domain`).
4. Add a `skill.yaml`, an example spec, and tests.

The optimization contract:

Method	Purpose
<code>requirements(spec)</code>	Understand + flag gaps
<code>formulations(spec)</code>	Enumerate LP/MILP/QUBO/Ising
<code>solve_classical_full(spec)</code>	The fair LP + MILP baseline
<code>reduce_to_instance(spec, max_qubits)</code>	Qubit-budgeted research instance
<code>build_qubo(instance, slack_bits)</code>	QUBO + documented encoding losses
<code>evaluate_bits(...)</code>	Decode bits → a verified <code>SolveResult</code>
<code>verify_full / verify_instance</code>	Exact constraint re-check
<code>instance_warm_start(...)</code>	Optional warm-start biases

Non-negotiable: `build_qubo` must declare its `encoding_losses`, and every decoded quantum solution must be re-checked against the exact constraints. That is what makes the verdict trustworthy. See [CONTRIBUTING.md](#).

Chapter 4

CLI reference

Eleven commands. Every command is deterministic (same spec + seed $\Rightarrow$ same result) and prints the disclaimer; nothing places trades or moves money.

```
qf-agent --help
qf-agent <command> --help
```

Command	Autonomy	Purpose
<code>solve</code>	L2 (L3 with <code>--approve</code>)	Run the full pipeline $\rightarrow$ decision + evidence bundle
<code>explain</code>	L0	Understand + formulate only. No solving
<code>plan</code>	L1	Experiment plan (reduction, backend, cost). No execution
<code>arena</code>	L2	Benchmark every family $\rightarrow$ honest leaderboard
<code>estimate</code>	L2	Quantum amplitude estimation vs classical MC
<code>simulability</code>	L2	Tensor-network classical-simulability check
<code>backends</code>	—	List backends + real availability
<code>skills</code>	—	List installed Quantum Skills
<code>runs</code>	—	List recorded runs from the evidence store
<code>serve</code>	—	Run the REST API + Studio
<code>version</code>	—	Version

4.1 solve

Runs all 9 agents and writes an evidence bundle.

```
qf-agent solve examples/collateral-allocation.yaml
qf-agent solve spec.yaml --out evidence/ --approve --quiet
```

Option	Default	Meaning
<code>--out, -o</code>	evidence	Evidence output directory
<code>--yes, --approve</code>	off	Approve paid/irreversible steps (required for real QPU)
<code>--quiet, -q</code>	off	Suppress the per-step trace

Exit codes: 0 success · 2 bad spec · 3 missing extra · 4 problem infeasible · 5 backend error.

Prints the FINAL DECISION block and writes `manifest.json`, `report.md`, `model_card.md`. A failed quantum step is recorded and the run **continues** on the classical baseline — it never silently aborts.

4.2 explain

L0 — validate the spec, enumerate candidate formulations (LP / MILP / QUBO / Ising) and what each can and cannot represent. No solving.

```
qf-agent explain examples/collateral-allocation.yaml
```

4.3 plan

L1 — the experiment plan: reduced instance size, QUBO density, backend target (or abstention + reasons), estimated depth/cost, and the **encoding losses the verifier will re-check**. No execution.

```
qf-agent plan examples/settlement-netting.yaml
```

4.4 arena

Benchmarks every bundled example across backends and prints the honest leaderboard. This is the headline artifact — see [FINDINGS.md](#).

```
qf-agent arena
qf-agent arena --out arena/      # also writes arena.md + arena.json
```

7 problems · 0 quantum advantage · 1 parity · 6 classical preferred.

4.5 estimate

Quantum Amplitude Estimation (MLAE, real Grover operator) vs exact summation and classical Monte Carlo, on a discretised loss distribution — plus an honest resource analysis. Needs the qiskit extra.

```
qf-agent estimate --qubits 4          # expected loss
qf-agent estimate --qubits 4 --tail 0.7 # VaR-style P(loss > 0.7)
qf-agent estimate --qubits 5 --out out/ # writes estimate.json
```

Option	Default	Meaning
--qubits, -m	4	Distribution qubits (2^m loss levels)
--tail	—	Tail threshold → exceedance probability; omit for expected loss
--shots	200	Shots per Grover power
--seed	7	Deterministic
--out, -o	—	Write estimate.json

4.6 simulability

Tensor-network baseline: is this problem’s QAOA circuit classically simulable by an MPS? Reports entanglement entropy, the bond dimension for a target fidelity, truncated-MPS fidelity, and the honest verdict. Needs qiskit; optimisation families only.

```
qf-agent simulability examples/collateral-allocation.yaml
qf-agent simulability spec.yaml --reps 2 --fidelity 0.999
```

4.7 backends

Every backend and whether it is available **right now** (truthful discovery — a credentialed backend reports exactly what is missing).

```
qf-agent backends
```

qaoa_sim	yes	qiskit statevector QA0A
qaoa_ibm	no	set QF_IBM_TOKEN (and optionally QF_IBM_INSTANCE/QF_IBM_BACKEND)
dwave_hybrid	no	install qf-agentos[dwave] (dwave-ocean-sdk)

4.8 skills

Installed Quantum Skills (a skill = a `skill.yaml` manifest declaring a problem family, its methods, and its verification checks).

```
qf-agent skills
qf-agent skills --dir ./my-skills
```

4.9 runs

List recorded runs from the file evidence store.

```
qf-agent runs
qf-agent runs --out evidence/
```

4.10 serve

Run the REST API and the bundled Studio. Needs the server extra. See [API.md](#).

```
qf-agent serve --host 0.0.0.0 --port 8000
# equivalently: uvicorn qf_agentos.api:app
```

4.11 version

```
qf-agent version
```

4.12 Running on real hardware

```
export QF_IBM_TOKEN='...' # your shell only; never committed
qf-agent backends          # confirm qaoa_ibm available=yes
qf-agent solve spec.yaml --approve # spec needs qpu_backend + autonomy L3
```

Real QPU execution requires **all** of: credentials, `qpu_backend: ibm|dwave`, `autonomy_level: L3`, sufficient `max_qpu_budget_usd`, and `--approve`. Missing any one falls back to the simulator with a recorded reason. Full runbook: [real-qpu.md](#).

Chapter 5

REST API reference

```
pip install -e ".[server]"
qf-agent serve --host 0.0.0.0 --port 8000    # or: uvicorn qf_agentos.api:app
```

Interactive docs at /docs (OpenAPI at /openapi.json). The bundled Studio is at /.

5.1 Endpoints

Method	Path	Auth	Purpose
GET	/healthz	open	Liveness + version
GET	/backends	open	Truthful backend capability discovery
GET	/skills	open	Installed Quantum Skills
GET	/examples	open	Bundled example specs (for the Studio)
GET	/	open	QF-Studio web UI
POST	/solve	key + rate-limit	Run the pipeline synchronously
POST	/jobs	key + rate-limit	Submit an async solve → job_id
GET	/jobs	key	List job records
GET	/jobs/{job_id}	key	Poll a job; returns the result when done
POST	/studio/run	key + rate-limit	Validate raw YAML + enqueue (Studio's entry point)
GET	/runs	key	The experiment registry
GET	/runs/{run_id}	key	A run's manifest

5.2 Authentication

Set QF_API_KEYS (comma-separated) to require an X-API-Key header on the solve endpoints and the registries:

```
export QF_API_KEYS='key-one,key-two'
curl -H 'X-API-Key: key-one' http://localhost:8000/runs
```

With no keys set the API is **open** (development mode) and logs a startup warning. Discovery endpoints and the Studio page stay open either way.

The three solve entry points are additionally **rate-limited** per client (QF_API_RATE_LIMIT_PER_MINUTE, default 60 → 429) and **size-guarded** (QF_API_MAX_INVENTORY, default 2000 → 413), across every problem family.

5.3 Synchronous solve

```
curl -X POST http://localhost:8000/solve \
  -H 'Content-Type: application/json' \
  -d '{"spec": { ... }, "approve": false, "include_report": true, "persist": true}'

{
  "run_id": "run-20260716T091948-18a20b9a",
  "decision": "CLASSICAL PREFERRED",
  "recommended_method": "classical_milp",
  "problem_infeasible": false,
  "objective_gap_pct": null,
  "evidence_digest": "646d5d51a249e6bd...",
  "warnings": ["No concentration limits given – a single issuer could dominate the pool."],
  "manifest": { "...": "... },
  "report_md": "# QF-AgentOS Evidence Report ..."
}
```

Field	Meaning
approve	Approve paid/irreversible steps (required for real QPU)
include_report	Include the Markdown evidence report
persist	Save the evidence bundle to the configured registry

5.4 Asynchronous solve (the production path)

/solve blocks. For long QAOA/QPU runs use the job queue:

```
# 1. submit → 202
curl -X POST http://localhost:8000/jobs -H 'Content-Type: application/json' \
  -d '{"spec": { ... } }'
# {"job_id": "d6b60f4cb67c...", "status": "queued"}

# 2. poll
curl http://localhost:8000/jobs/d6b60f4cb67c...

{
  "job_id": "d6b60f4cb67c...",
  "status": "succeeded",
  "problem": "collateral_allocation",
  "created_at": 1.0, "started_at": 1.1, "finished_at": 6.2,
  "error": null,
  "result": { "decision": "CLASSICAL PREFERRED", "...": "..." }
}
```

Status: queued → running → succeeded | failed (with error). Jobs run in a bounded thread pool (QF_API_JOB_WORKERS, default 2) with an LRU-capped registry (QF_API_MAX_JOBS, default 256). In-memory and single-instance by design — for multi-replica deployments put a real broker behind the same interface.

5.5 Studio entry point

POST /studio/run takes **raw YAML** (what humans write), validates it with the real validator, and enqueues an async job:

```
curl -X POST http://localhost:8000/studio/run -H 'Content-Type: application/json' \
  -d '{"spec_yaml": "problem: collateral_allocation\nconstraints: {...}", "approve": false}'
# → 202 {"job_id": "...", "status": "queued"} then poll GET /jobs/{id}
```

Invalid YAML, a non-mapping document, or a spec that fails validation returns **422** with an actionable message. Malformed or too-deeply-nested requests return **400** — never a 500.

5.6 Error codes

Code	Meaning
400	Malformed / too deeply nested request
401	Missing or invalid X-API-Key
413	Problem exceeds QF_API_MAX_INVENTORY — use the CLI/SDK
422	Invalid spec (validation message included)
429	Rate limit exceeded (Retry-After: 60)
500	Pipeline produced no decision (should not happen)

5.7 QF-Studio

Two frontends, same API:

- **Bundled** (GET /) — a self-contained vanilla-JS page, ships in the wheel, no build step, no external requests. Pick or paste a spec → solve → colour-coded verdict badge, metrics, warnings, rendered evidence report, recent-runs table.
- **React** (`studio-react/`) — a richer Vite + TypeScript SPA kept as a separate deployable so the Python package stays Node-free.

```
cd studio-react && npm install && npm run dev    # proxies the API on :8000
```

5.8 Registries

Runs persist through one pluggable interface (QF_REGISTRY_BACKEND):

Backend	Config	Use
file (default)	QF_EVIDENCE_DIR	Dependency-free, single host
mlflow	QF_MLFLOW_TRACKING_URI, QF_MLFLOW_EXPERIMENT	Shared experiment tracking
postgres	QF_POSTGRES_DSN	Multi-instance SQL persistence

See [CONFIGURATION.md](#).

5.9 Docker

```
docker compose up --build    # API + Studio on :8000, persists evidence/
```

Chapter 6

Configuration

All settings are environment variables with the **QF_** prefix, read once per process (see [.env.example](#)). Credentials are held as SecretStr — never logged, never printed in a repr, never written into an evidence bundle.

```
export QF_LOG_FORMAT=json
export QF_REGISTRY_BACKEND=mlflow
qf-agent solve spec.yaml
```

6.1 Observability

Variable	Default	Meaning
QF_LOG_LEVEL	INFO	DEBUG / INFO / WARNING / ERROR / CRITICAL
QF_LOG_FORMAT	text	text or json (run-id-correlated structured logs)
QF_TRACING_ENABLED	false	OpenTelemetry spans per agent step (needs the otel extra)

6.2 Determinism & numerics

Variable	Default	Meaning
QF_DEFAULT_SEED	7	Default seed (a spec's execution_policy.seed overrides)
QF_NUMERIC_REL_TOL	1e-6	Relative tolerance for objective comparisons

Same spec + same seed $\Rightarrow$ byte-identical evidence_digest.

6.3 Simulation budgets

Guard-rails against runaway resource use. The planner **abstains** (honestly) rather than exceeding them.

Variable	Default	Meaning
QF_STATEVECTOR_QUBIT_LIMIT	22	Max qubits for exact statevector QAOA
QF_BRUTEFORCE_QUBIT_LIMIT	20	Max qubits for brute-force QUBO ground truth

6.4 Governance / registry

Variable	Default	Meaning
QF_EVIDENCE_DIR	evidence	File-store root for evidence bundles
QF_REGISTRY_BACKEND	file	file · mlflow · postgres
QF_MLFLOW_TRACKING_URI	—	MLflow tracking URI (default ./mlruns); needs the mlflow extra
QF_MLFLOW_EXPERIMENT	qf-agentos	MLflow experiment name
QF_POSTGRES_DSN	—	e.g. postgresql+psycopg://user:pass@host/db; needs the postgres extra

```
# MLflow
export QF_REGISTRY_BACKEND=mlflow QF_MLFLOW_TRACKING_URI=http://mlflow:5000
# Postgres
export QF_REGISTRY_BACKEND=postgres QF_POSTGRES_DSN='postgresql+psycopg://qf:pw@db/qf'
```

QF_REGISTRY_BACKEND=postgres without a DSN fails fast with a clear error.

6.5 REST API

Variable	Default	Meaning
QF_API_KEYS	"" (open)	Comma-separated accepted X-API-Key values
QF_API_RATE_LIMIT_PER_MINUTE	60	Per-client limit on the solve endpoints
QF_API_MAX_INVENTORY	2000	Max problem size over the API (all families) → 413
QF_API_JOB_WORKERS	2	Concurrent async solve workers
QF_API_MAX_JOBS	256	Retained job records (LRU)

With QF_API_KEYS unset the API is **open** and logs a startup warning. Always set it in any shared deployment.

6.6 Credentials (secrets)

Never commit these or paste them anywhere. Export them in your own shell.

Variable	Default	Meaning
QF_IBM_TOKEN	—	IBM Quantum API key → enables qaoa_ibm
QF_IBM_CHANNEL	ibm_quantum_platform	IBM channel; or ibm_cloud. (<i>The legacy ibm_quantum channel was retired.</i>)
QF_IBM_INSTANCE	—	Instance CRN / hub-group-project (optional; auto-selected on the Open plan)
QF_IBM_BACKEND	—	Pin a device; otherwise least-busy is chosen
QF_DWAVE_TOKEN	—	D-Wave Leap token → enables dwave_hybrid

Credentials alone are **not** sufficient to run a QPU. Real hardware also requires qpu_backend: ibm|dwave in the spec, autonomy_level: L3, a sufficient max_qpu_budget_usd, and explicit --approve. See [real-qpu.md](#).

6.7 Spec-level policy (per run, not env)

Set in the spec's execution_policy — these travel with the problem and into the evidence bundle:


```

execution_policy:
  compare_classical: true
  allow_gate_model: true
  allow_quantum_annealing: true
  qpu_backend: sim          # sim | ibm | dwave
  autonomy_level: L2        # L0...L4
  max_qpu_budget_usd: 0
  max_effective_qubits: 20
  qaoa_reps: 1
  shots: 4096
  seed: 7
  noisy_simulation: false   # + noise_two_qubit_error, readout_error

```

6.8 Extras

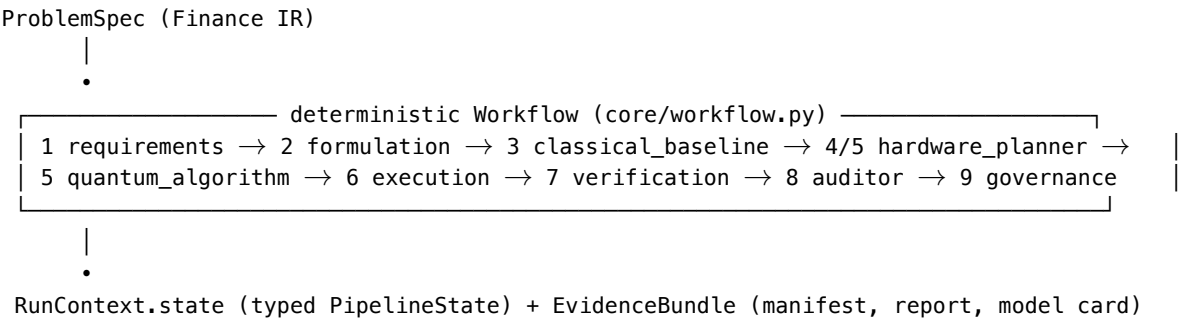
Extra	Installs	Enables
(none)	scipy, pydantic	Classical MILP/LP, the full pipeline
qiskit	qiskit, qiskit-aer	qaoa_sim, noisy sim, QAE, tensor-network analysis
ibm	+ qiskit-ibm-runtime	qaoa_ibm (real hardware)
dwave	dwave-ocean-sdk	dwave_hybrid (real annealer)
pennylane	pennylane	qaoa_pennylane
server	fastapi, uvicorn	REST API + Studio
mlflow / postgres	mlflow / sqlalchemy+psycpg	Registries
otel	opentelemetry	Tracing
all / dev	everything / + test tooling	

Chapter 7

Architecture

QF-AgentOS turns a financial optimisation problem into a verified, evidence-backed decision about *whether and how* to use quantum computing. This document describes the runtime, the agent contract, the backend abstraction, and the safety model.

7.1 High-level flow



The workflow is a fixed, ordered list of typed steps. Determinism is a feature: the same spec + seed produce the same `evidence_digest`. Steps are **isolated** — a failing agent is recorded as a `StepError` and the run continues so partial evidence is still emitted.

7.2 The agent contract

Each agent is `Callable[[RunContext], str]`. It reads and writes the **typed** blackboard `ctx.state` (`core/state.py::PipelineState`) — never a stringly-typed dict — and returns a one-line human summary. Artifacts exchanged between agents are Pydantic models in `core/artifacts.py`, so the inter-agent contract is mypy-checked and serialises cleanly into the evidence bundle.

#	Agent	Reads	Writes
1	requirements	spec	requirements
2	formulation	spec	formulations
3	classical_baseline	spec	classical_lp, classical_milp
4/5	hardware_planner	spec	instance, qubo, hardware_plan
5	quantum_algorithm	hardware_plan	quantum_selection
6	execution	instance, qubo, hardware_plan	instance_milp, instance_qubo_exact, instance_sa, instance_qaoa, qaoa_raw
7	verification	all results	verification, reproducibility
8	auditor	verification + results	audit
9	governance	everything	bundle

7.3 The three formulations (fair comparison)

The same economic problem is expressed so classical and quantum are compared like-with-like:

- **Continuous LP** — theoretical lower bound on cost.
- **Binary MILP** — the *fair* classical comparator (scipy/HiGHS, exact).
- **QUBO** → **Ising** — a reduced “research instance” for the quantum backend.

The QUBO encodes the coverage requirement as a proper inequality ($\text{cov} \geq R'$) using binary **slack bits**, with an **adaptive penalty** that guarantees the QUBO ground state respects coverage. Concentration and HQLA are *not* encoded — they become **verification-only** constraints. The Verification agent re-checks every decoded quantum solution against the *full* constraint set, so relaxation artifacts are caught, not hidden.

7.4 Backends (backends/)

All QUBO solvers implement the QuboSolver protocol (backends/base.py):

```
class QuboSolver(Protocol):
    name: str
    kind: str # classical | heuristic | quantum
    requires_credentials: bool
    def is_available(self) -> tuple[bool, str]: ...
    def solve(self, qubo: Qubo, config: QuboRunConfig) -> QuboSolution: ...
```

The registry is the single source of truth for discovery and instantiation:

backend	kind	credentials	notes
qubo_exact_optimum	classical	no	brute force ($\leq$ brute_force_qubit_limit)
simulated_annealing	heuristic	no	multi-restart SA
qaoa_sim	quantum	no	Qiskit statevector QAOA (default path)
qaoa_pennylane	quantum	no	PennyLane alternative simulator
qaoa_ibm	quantum	yes	IBM Runtime; optimise on sim, sample on QPU
dwave_hybrid	quantum	yes	D-Wave Leap hybrid sampler

Hardware backends are inert without their SDK + credentials (they raise BackendUnavailableError) and are gated behind autonomy **L3** + approval.

7.5 Safety model (core/policy.py)

Autonomy levels L0–L4 gate every side-effectful action:

Action	Min level	Extra gate
explain	L0	—
plan	L1	—
run_simulator	L2	—
run_paid_qpu	L3	budget + human approval
recommend_production	L4	human approval

The gates have real call sites: the Execution agent authorises RUN_SIMULATOR/RUN_PAID_QPU before running a backend (chosen by requires_credentials), and the Auditor authorises RECOMMEND_PRODUCTION. The system never places trades, moves money, runs paid QPUs without approval, or claims quantum advantage without passing deterministic verification.

7.6 Configuration & observability

- `core/config.py` — environment-driven Settings (prefix `QF_`); credentials as `SecretStr` (never logged or serialised).
- `core/observability.py` — structured logging correlated by `run_id`, with an optional JSON format for ingestion.

7.7 Governance (governance/)

- `report.py` — builds the manifest, technical report, and model card, plus a deterministic `evidence_digest` (over decision-relevant content, excluding timestamps) for reproducibility checks.
- `store.py` — file-based experiment registry (`index.jsonl` + per-run bundle); swappable for a database behind the same interface.

7.8 Surfaces

- **CLI** (`cli.py`): `solve`, `explain`, `plan`, `skills`, `runs`, `serve`, `version`.
- **REST API** (`api.py`): `/healthz`, `/backends`, `/skills`, `/solve`, `/runs`.
- **SDK**: `from qf_agentos import load_spec, solve`.

7.9 Extending

See [CONTRIBUTING.md](#) for how to add a Quantum Skill, a backend, or a new problem family.

Chapter 8

Running on real quantum hardware

QF-AgentOS can send the **final optimised QAOA circuit** to real IBM hardware via Qiskit Runtime. The variational parameters are trained on the local statevector simulator (free, fast); only the single best circuit is sampled on the device. This keeps QPU usage — and cost — minimal.

Honesty note. Real-hardware results are noisy and re-verified against the exact constraints like every other backend. A device run that fails (queue, transient error, bad credentials) is recorded and the run completes on the classical baseline — quantum is never faked.

8.1 Prerequisites

```
pip install 'qf-agentos[ibm]' # qiskit + qiskit-aer + qiskit-ibm-runtime
```

You need an **IBM Quantum Platform** account and an API key (<https://quantum.ibm.com/>). The free **Open Plan** works — QF-AgentOS samples the final circuit in **job execution mode**, which the Open plan allows (it forbids interactive *session* mode).

8.2 Provide the credential — as an environment variable, never in a file

QF-AgentOS reads the token from the environment as a `SecretStr`; it is never logged, printed, or written into an evidence bundle. **Set it in your own shell — do not commit it or paste it anywhere:**

```
export QF_IBM_TOKEN='your-api-key'
# Optional:
export QF_IBM_CHANNEL='ibm_quantum_platform' # default; or 'ibm_cloud'
export QF_IBM_INSTANCE='<your-instance-CRN>' # if your account needs one
export QF_IBM_BACKEND='ibm_brisbane' # pin a device; else least-busy
```

The legacy `ibm_quantum` channel was retired in `qiskit-ibm-runtime` 0.40+. This platform defaults to `ibm_quantum_platform`; set `QF_IBM_CHANNEL` if your account uses `ibm_cloud`.

Check it is visible to the backend registry:

```
qf-agent backends # 'qaoa_ibm' should show available=true
```

8.3 Opt in, and approve the paid run

Real hardware is gated: it requires autonomy **L3** and explicit human approval and a sufficient budget. Add to your spec:

```
execution_policy:
  qpu_backend: ibm # route the final circuit to IBM (default: sim)
```

```

autonomy_level: L3      # RUN_PAID_QPU requires L3
max_gpu_budget_usd: 5    # must cover the estimated cost (Open Plan is $0)
max_effective_qubits: 12

```

Then run **with explicit approval**:

```
qf-agent solve examples/collateral-allocation.yaml --approve
```

Without `--approve` the planner still routes to IBM, but the executor refuses the paid run and the pipeline continues on the simulator (you'll see a warning). If credentials are missing, it falls back to the simulator with a recorded reason.

8.4 What you get

The evidence bundle records the real device name, the QPU time, and the decoded solution re-verified against the full constraints — alongside the classical MILP and the ideal simulator, so the comparison stays honest.

8.5 D-Wave (annealing)

The same pattern works for the D-Wave Leap hybrid annealer, which solves the QUBO directly (no gate-model circuit):

```

pip install 'qf-agentos[dwave]'
export QF_DWAVE_TOKEN='your-leap-token' # free Leap tier: 1 min QPU/month
qf-agent backends                        # dwave_hybrid should show available=true

```

```

execution_policy:
  gpu_backend: dwave      # route the QUBO to the annealer
  autonomy_level: L3
  max_gpu_budget_usd: 5

```

```
qf-agent solve examples/collateral-allocation.yaml --approve
```

Same gating (L3 + approval + budget), same honest re-verification; falls back to the simulator with a recorded reason if D-Wave is unavailable.

Chapter 9

Research note — QAE & tensor networks

Author: Quantum Finance Scientist (QF-AgentOS) **Date:** 2026-07-16 **Status:** Working draft

Two research-grade quantum-finance extras were added to QF-AgentOS, each chosen because it sharpens the platform's core question — *does quantum actually help?* — rather than blurring it. This note records the methodology, the honest complexity analysis, and the verdict for both.

Chapter 10

Part A — Quantum Amplitude Estimation for risk

Implementation: `qf_agentos/finance/qae.py`; CLI: `qf-agent estimate`.

10.1 Problem statement

Estimate an expectation under a discretised distribution:

$$\mathbb{E}[f(X)] = \sum_{i=0}^{2^m-1} p_i f_i, \quad f_i \in [0, 1].$$

Concrete instances shipped: the **expected loss** of a discretised truncated-normal loss distribution over 2^m levels, and a **VaR-style exceedance probability** $\Pr[\text{loss} > t]$ (payoff = tail indicator). Both are the canonical targets of quantum risk analysis (Woerner & Egger 2019) and derivative pricing (Stamatopoulos et al. 2020).

10.2 Classical baseline

Best classical method: For the small discretised distributions a statevector can hold ($m \leq \sim 12$), the expectation is an **exact sum** — $O(2^m)$, instant, zero error. For continuous/high-dimensional versions the workhorses are **Monte Carlo** ($O(1/\epsilon^2)$ samples for RMSE ϵ) and its stronger cousins **quasi-MC** (Sobol) and **multilevel MC** (Giles) — all production-scale. **Why this is the right baseline:** any quantum claim must beat *exact summation* at these sizes and *MC/QMC/MLMC* asymptotically — not a naive comparator (guardrail G-baseline).

10.3 Quantum approach

Primitive: Quantum Amplitude Estimation, specifically **Maximum-Likelihood AE** (Suzuki et al. 2020) — chosen over canonical (QPE-based) QAE because it avoids deep phase-estimation circuits, and over a raw single-shot estimate because MLAE resolves the amplitude-aliasing across Grover powers.

Algorithm: MLAE for $\mathbb{E}[f]$

Input: probabilities p , payoff f in $[0,1]$, Grover powers $\{k\}$, shots N

Output: estimate of $a = \mathbb{E}[f]$

1. Build A : $|0\rangle \rightarrow \sum_i \sqrt{p_i} |i\rangle (\sqrt{1-f_i} |0\rangle + \sqrt{f_i} |1\rangle)$ # good-state prob == a
2. Grover operator $Q = -A S_0 A^\dagger$ (0 marks objective qubit = $|1\rangle$)
3. For each power k : $P_k = \langle \text{good} | Q^k A | 0 \rangle = \sin^2((2k+1)\theta)$; $h_k \sim \text{Binomial}(N, P_k)$
4. $\theta_{\text{hat}} = \text{argmax}_{\theta} \sum_k [h_k \log \sin^2((2k+1)\theta) + (N-h_k) \log \cos^2(\dots)]$
5. return $a_{\text{hat}} = \sin^2(\theta_{\text{hat}})$

The amplitude oracle is built and **validated exactly**: the objective-qubit good-state probability equals $\sum_i p_i f_i$ to machine precision (a unit test enforces this). The MLAE loop uses a real Grover operator on the statevector; only the shot sampling is stochastic.

10.4 Complexity analysis

Cost component	Classical (MC)	Quantum (QAE)
Per-query	$O(1)$ sample	$O(1)$ oracle call
State preparation	—	$O(2^m)$ (arbitrary distribution)
Queries for RMSE ϵ	$O(1/\epsilon^2)$	$O(1/\epsilon)$
Readout	—	$O(1)$ (single amplitude)
End-to-end	$O(1/\epsilon^2)$	$O(2^m + 1/\epsilon)$

Speedup status: **Proven** quadratic in queries (Brassard, Høyer, Mosca, Tapp 2002; Montanaro 2015). **Assumptions:** amplitude-oracle access — i.e. an efficient state-preparation unitary. For an *arbitrary* financial distribution this costs $O(2^m)$ and is the binding constraint (guardrail G2/G3): the quadratic advantage survives only when state prep is amortised across many pricings (same distribution, many strikes) (Stamatopoulos et al. 2020) or the distribution is efficiently loadable (Grover–Rudolph for log-concave; qGAN, Zoufal et al. 2019).

10.5 Hardware feasibility

Required regime: early fault tolerance. The quadratic advantage requires large Grover powers $\rightarrow$ deep *coherent* circuits; NISQ noise destroys them at useful precision. **Circuit depth:** $\propto \max_k (2k+1) \cdot \text{depth}(A)$, with $\text{depth}(A)$ itself $O(2^m)$ for exact loading. **Realistic horizon:** the threshold for a genuine advantage in derivative pricing is estimated at thousands of logical qubits and very deep circuits (Chakrabarti et al. 2021) — a 10+ year horizon.

10.6 Empirical result (this implementation)

On a 4-qubit expected-loss instance (qf-agent estimate --qubits 4): MLAE recovers the exact expectation to $\sim 10^{-3}$, but **classical MC at the same query budget matches or beats it**, and exact summation ($2^4 = 16$ terms) is instant and error-free. The resource analysis reports the proven $1/\epsilon$ vs $1/\epsilon^2$ ratio while flagging the $O(2^m)$ state-prep bottleneck. **Verdict: CLASSICAL PREFERRED** at every size the platform can run.

10.7 Open questions

1. State preparation competitive with the QAE inner loop (closing the end-to-end gap).
2. QAE + multilevel-MC coupling — can they compose below either alone?
3. Error-mitigated QAE that preserves the quadratic scaling.

Chapter 11

Part B — Tensor-network classical-simulability baseline

Implementation: `qf_agentos/finance/tensor_network.py`; CLI: `qf-agent simulability`.

11.1 Problem statement

Given the QAOA output state $|\psi\rangle$ for a reduced portfolio/collateral/routing QUBO, decide: **is this quantum circuit classically simulable by a tensor network?** If a matrix-product state (MPS) of modest bond dimension χ reproduces $|\psi\rangle$, then a classical algorithm with $O(n\chi^2)$ resources matches the circuit — undercutting any quantum-advantage claim from that instance.

11.2 Classical baseline

Method: MPS / DMRG-style simulation. An n -qubit state has an exact MPS with bond dimension up to $2^{\lfloor n/2 \rfloor}$; the *useful* regime is when the entanglement across every cut is low, so χ stays small and simulation is polynomial (Vidal 2003). This is simultaneously a classical *competitor* and a diagnostic.

11.3 Method

For each bipartition, compute the Schmidt spectrum (SVD of the reshaped amplitude tensor), the bipartite von-Neumann entropy $S = -\sum_i \lambda_i^2 \log_2 \lambda_i^2$, and the bond dimension χ_ϵ needed to capture fidelity $1 - \epsilon$. Then reconstruct a truncated bond- χ MPS and report its fidelity $|\langle \psi_{\text{MPS}} | \psi \rangle|^2$. Validated on product ($S = 0, \chi = 1$), Bell/GHZ ($S = 1, \chi = 2$), and random ($\chi = 2^{n/2}$ for exact recovery) states.

11.4 Complexity

Representation	Parameters	Simulable when
Statevector	2^n	always (small n)
MPS (bond χ)	$O(n\chi^2)$	$\chi \ll 2^{n/2}$ (low entanglement)

11.5 Empirical result

The shipped dense-QUBO instances produce **highly entangled** QAOA states — e.g. the 13-qubit collateral instance has max bipartite entropy ≈ 5.2 bits and needs $\chi \approx 43$ against an exact-rank max of 64. So an MPS gives *no* compression here; the analysis reports this honestly as a **small-instance artefact, not hardware advantage** (exact statevector already solves it classically). The value delivered is a *quantified*

entanglement/compressibility measure and an honest crossover statement: MPS-simulable (low- χ) circuits are trivially classical; high- χ ones still fall to exact statevector at these sizes.

11.6 Caveats

- Dense (all-to-all) QUBOs generate near-maximal entanglement — MPS is the wrong classical tool for them; the honest baseline there is exact enumeration / MILP (already the platform’s).
 - The analysis uses the exact statevector, so it *quantifies* rather than *replaces* classical simulation — which is the intended, honest role.
-

11.7 Overall conclusion

Neither extra produces a quantum advantage on any instance QF-AgentOS can run — and both are engineered to *say so with evidence*. QAE’s advantage is proven but asymptotic and state-prep-bottlenecked (early-FT horizon); the tensor-network baseline quantifies exactly when a QAOA circuit is classically simulable. Both are faithful to the platform’s thesis: an agent that runs the quantum method honestly and reports, with proof, that classical still wins.

11.8 References

- Brassard, Høyer, Mosca, Tapp 2002. *Quantum amplitude amplification and estimation*.
- Montanaro 2015. *Quantum speedup of Monte Carlo methods*. Proc. R. Soc. A.
- Suzuki et al. 2020. *Amplitude estimation without phase estimation*. Quantum Inf. Process.
- Grinko et al. 2021. *Iterative quantum amplitude estimation*. npj Quantum Information.
- Woerner, Egger 2019. *Quantum risk analysis*. npj Quantum Information.
- Stamatopoulos, Egger et al. 2020. *Option pricing using quantum computers*. Quantum.
- Zoufal, Lucchi, Woerner 2019. *qGANs for learning and loading random distributions*.
- Chakrabarti et al. 2021. *A threshold for quantum advantage in derivative pricing*. Quantum.
- Vidal 2003. *Efficient classical simulation of slightly entangled quantum computations*. PRL.

Research artifact — decision-support only, not investment advice.